

毕业设计中期报告

Rust异步驱动模块设计与实现

汇报人：黄昊颖 指导老师：向勇

目录

Contents

研究内容

01

步骤与进展

02

问题与拟解决方案

03

下一步研究任务

04

进度安排

05

参考资料与日志

06



01.

研究内容

研究内容

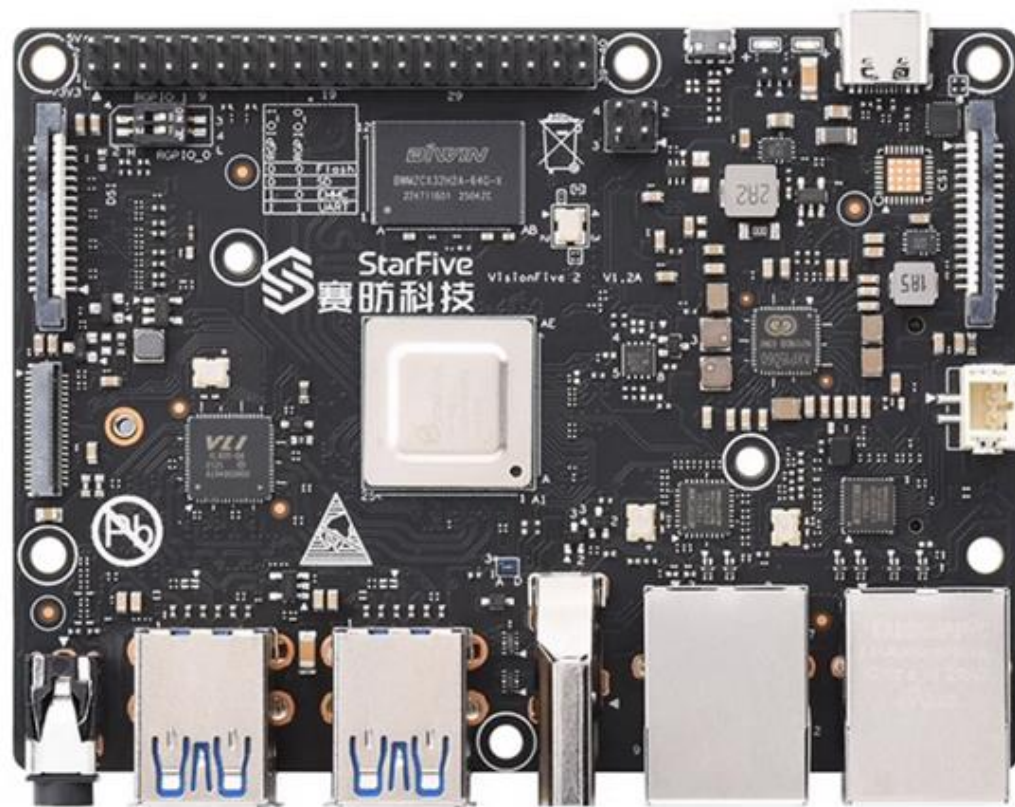
主要内容

开发 Rust Future 异步硬件驱动模块

预期成果

支持以下的Rust异步驱动模块

- 1) VisionFive2星光2
串口外设
- 2) Flash闪存块设备
SD卡



昉·星光2 开发板



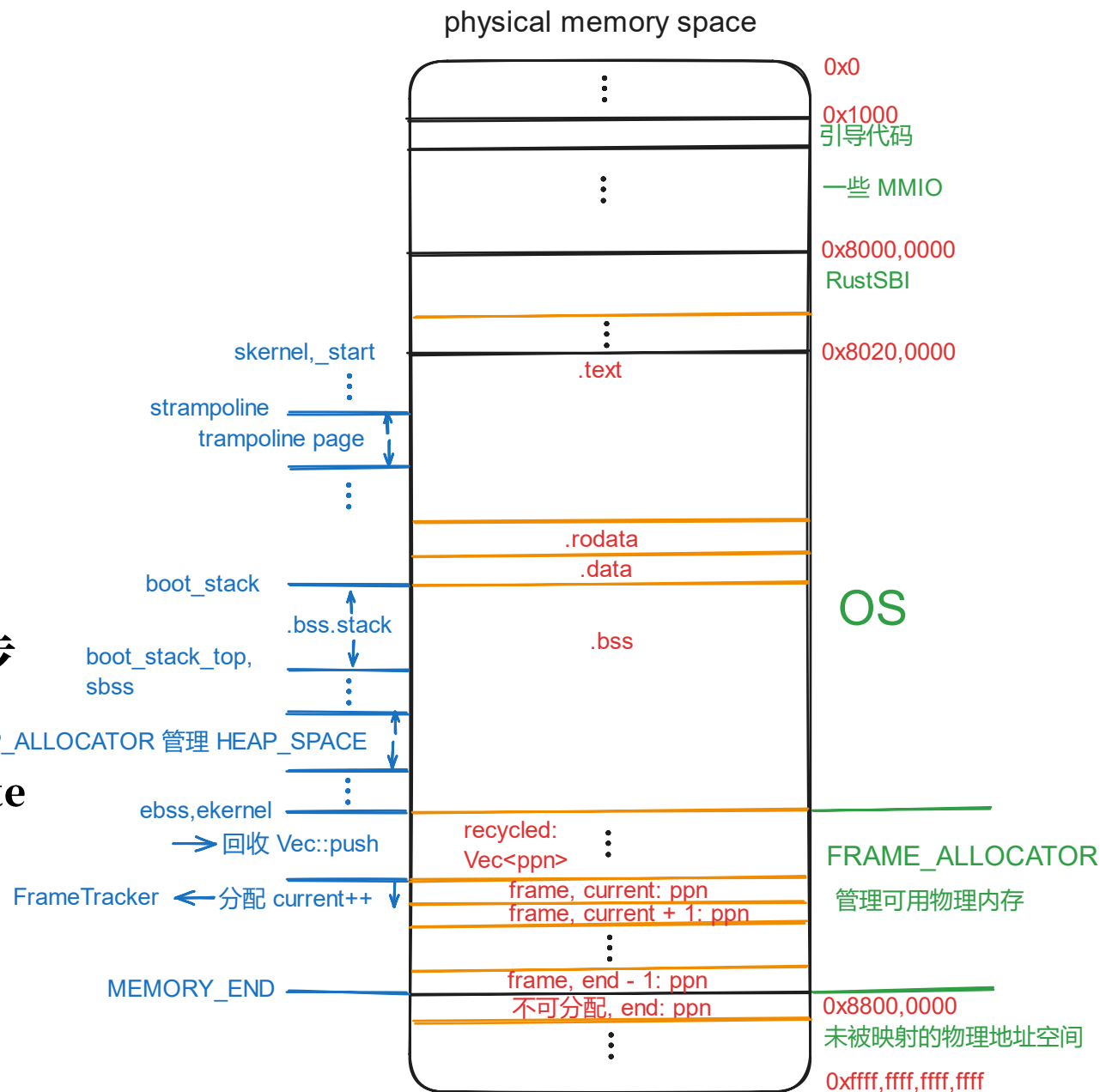
02.

实验步骤与目前进展

实验步骤与目前进展

实验步骤

- 1) rCore-N 环境配置
- 2) rCore-N 异步 timer 开发
 - 2.1) 参考 embassy timer 实现
 - 2.2) 原 timer 逻辑 Rust Future 重写
- 3) 移除 rCore-N 原串口逻辑引入异步
 - 3.1) 引入林晨 async-uart-driver
 - 3.2) 或者自己实现一个异步串口独立 crate
- 4) 步骤3) 移植到星光二开发板
- 5) 基于以上逻辑开发异步块设备驱动



rCore/QEMU 物理地址空间

实验步骤与目前进展

目前进展：rCore-N

1) rCore-N 简介

- 1.1) 基于 rCore 并支持用户态 CSR 指令
- 1.2) 只有进程模型无线程模型
- 1.3) 实现了自己的串口驱动逻辑

2) rCore-N 环境配置：已完成

- 2.1) 带 N 扩展的 QEMU 成功编译
- 2.2) rCore-N 编译运行内核启动
- 2.3) 跑 uart_benchmark U 态串口测例

```
[DEBUG 0]: boot_stack 0x80534000 top 0x80574000
[DEBUG 0]: trying to add initproc
[DEBUG 0]: add_initproc
[DEBUG 0]: initproc added to task manager!
[hart 0]satp: 0x80000000000080775, sp: 0x80543ed0
[DEBUG 0]: [kernel 0] Start 1
[DEBUG 0]: [kernel 0] Start 2
[hart 1]satp: 0x80000000000080775, sp: 0x80553ed0
[hart 2]satp: 0x80000000000080775, sp: 0x80563ed0
[DEBUG 0]: [kernel 0] Start 3
[hart 1]Hello
[hart 2]Hello
[hart 0]Hello
[hart 3]satp: 0x80000000000080775, sp: 0x80573ed0
```

```
/dev/pts/1
Rust user shell
>> uart_benchmark
[uart benchmark] User mode interrupt driver benchm
[uart load] trap init result: 0xffffffffffffd000,
[uart load] trap init result: 0xffffffffffffd000,
[uart load] Interrupt mode, claim result: 0x100040
[uart load] Interrupt mode, claim result: 0x100050
[uart load] Intr count: 9, Tx: 9, Rx: 0, err pos:
[uart 15] Test finished, 16 bytes sent, 0 bytes re
[uart load] Intr count: 9, Tx: 9, Rx: 0, err pos:
[uart 14] Test finished, 16 bytes sent, 0 bytes re
[uart benchmark] User mode interrupt driver benchm
Shell: Process 1 exited with code 0
>> _
```

rCore-N 异步串口驱动模块运行记录

实验步骤与目前进展

目前进展：Embassy

Embassy 学习：运行 Embassy 示例程序并追踪程序执行流

- 1.1) embassy-executor 状态 State 转移
- 1.2) embassy timer 设计
- 1.3) embassy 主要数据结构之间的转换
各个数据结构的设计与访问并非孤立而是互相联系
- 1.4) 基于中断的 executor

实验步骤与目前进展

目前进展: Embassy

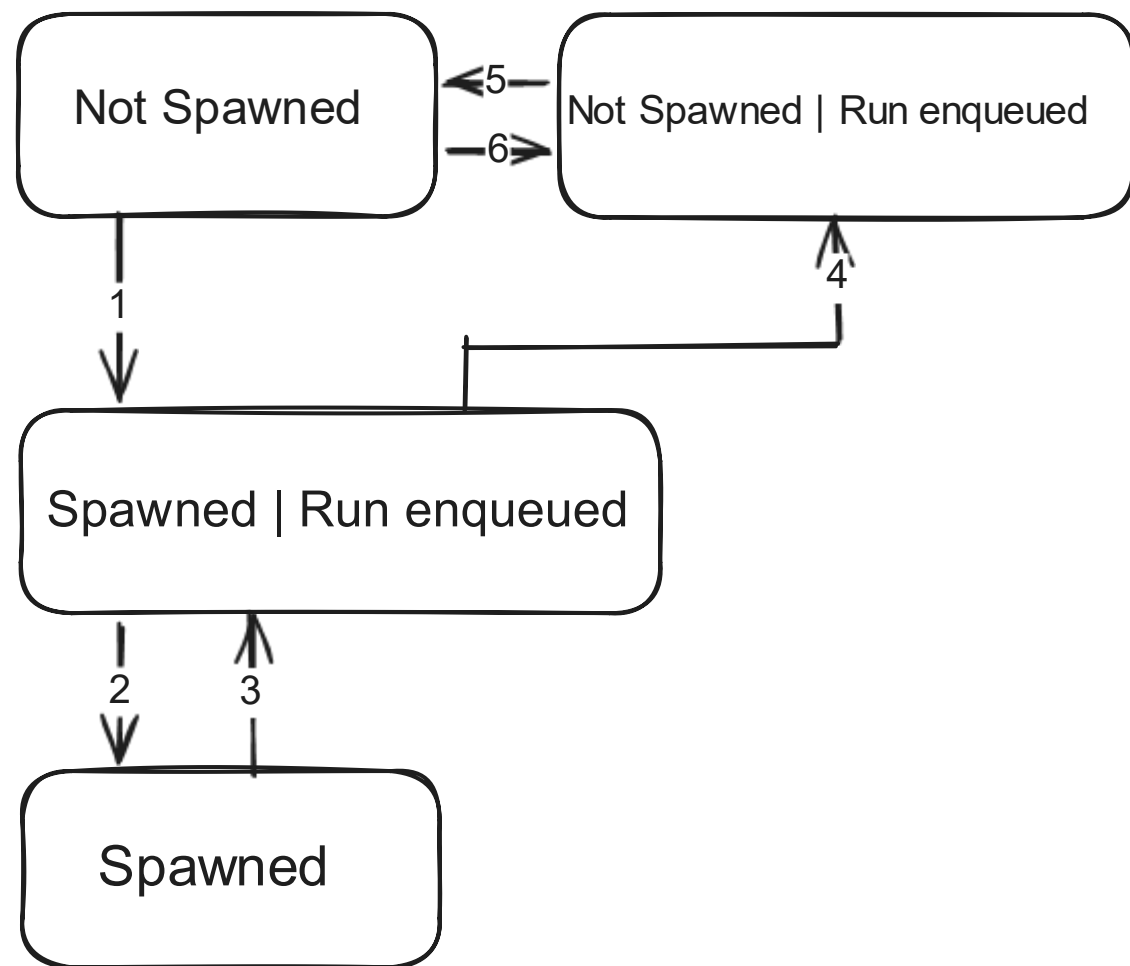
1) embassy-executor 状态 State 转移

1.1) **Not Spawned** 初始状态 TaskStorage 槽位为空没有正在运行的 future

1.2) **Spawned | Run enqueued** 任务 future 存在且在 RunQueue 中等待下次 poll() 执行 poll_fn

1.3) **Spawned** 任务 future 存在但不在 RunQueue 中所以不会马上被 poll

1.4) **Not spawned | Run enqueued** 此时 poll_fn 已经被换成空的 poll_exited() 了, 任务已经执行结束了但还在 RunQueue 中等下次 poll() 被清理



来自 embassy-executor/src/raw/mod.rs

实验步骤与目前进展

目前进展：Embassy

1) embassy-executor 状态 State 转移

状态转移	状态修改	备注
1	State::spawn()	spawn 一个任务, 任务 future 在 TaskPool 中某个 TaskStorage 槽位中
2	State::run_dequeue()	后续把任务从 RunQueue 中拿出 poll 执行 poll_fn
3	State::run_enqueue()	任务被 wake 重新放入 RunQueue 等待下一次 poll 执行
4	State::despawn()	任务执行完成后等待下一次 poll 被清出 RunQueue
5	State::run_dequeue()	后续把结束任务从 RunQueue 中拿出 poll 执行空的 poll_exited
6	State::run_enqueue()	已经退出的任务收到迟到的 wake 被短暂地重新放进 RunQueue, 等下一轮 poll 时清出 RunQueue

实验步骤与目前进展

目前进展: Embassy

2) embassy timer 设计

2.1) 维护一个 timer queue 链表 embassy-time-queue-utils::Queue

2.2) Queue::schedule_wake(at, waker) 通过 waker 在 timer queue 中找对应的项, 如果找到且发现 at 会提前唤醒时间则更新项, 如果未找到则向列表中插入该新项

2.3) Queue::next_expiration() 遍历 timer queue 链表中所有项, 对超时项进行 wake_task() 并踢出 timer queue 链表。wake_task() 会将当前任务对应项加入到 RunQueue 中触发 __pender() 函数触发 SyncExecutor::poll()

2.4) embassy-rp 实现了自己的 TimerDriver, 根据 embassy 规范 TimerDriver 是一个 Driver trait 需要实现 2 个 trait 方法 now() 和 schedule_wake()

2.5) embassy-rp 在 TimerDriver 中还实现了其它 3 个辅助函数。set_alarm() 硬件设置下一次时钟中断到来的时间并记录该次的 timestamp; 中断到来调用 check_alarm() 检查记录的 timestamp 是否过期若过期则调用 trigger_alarm(); trigger_alarm() 调用 Queue::next_expiration() 并 set_alarm() 设置下一次硬件中断

实验步骤与目前进展

目前进展: Embassy

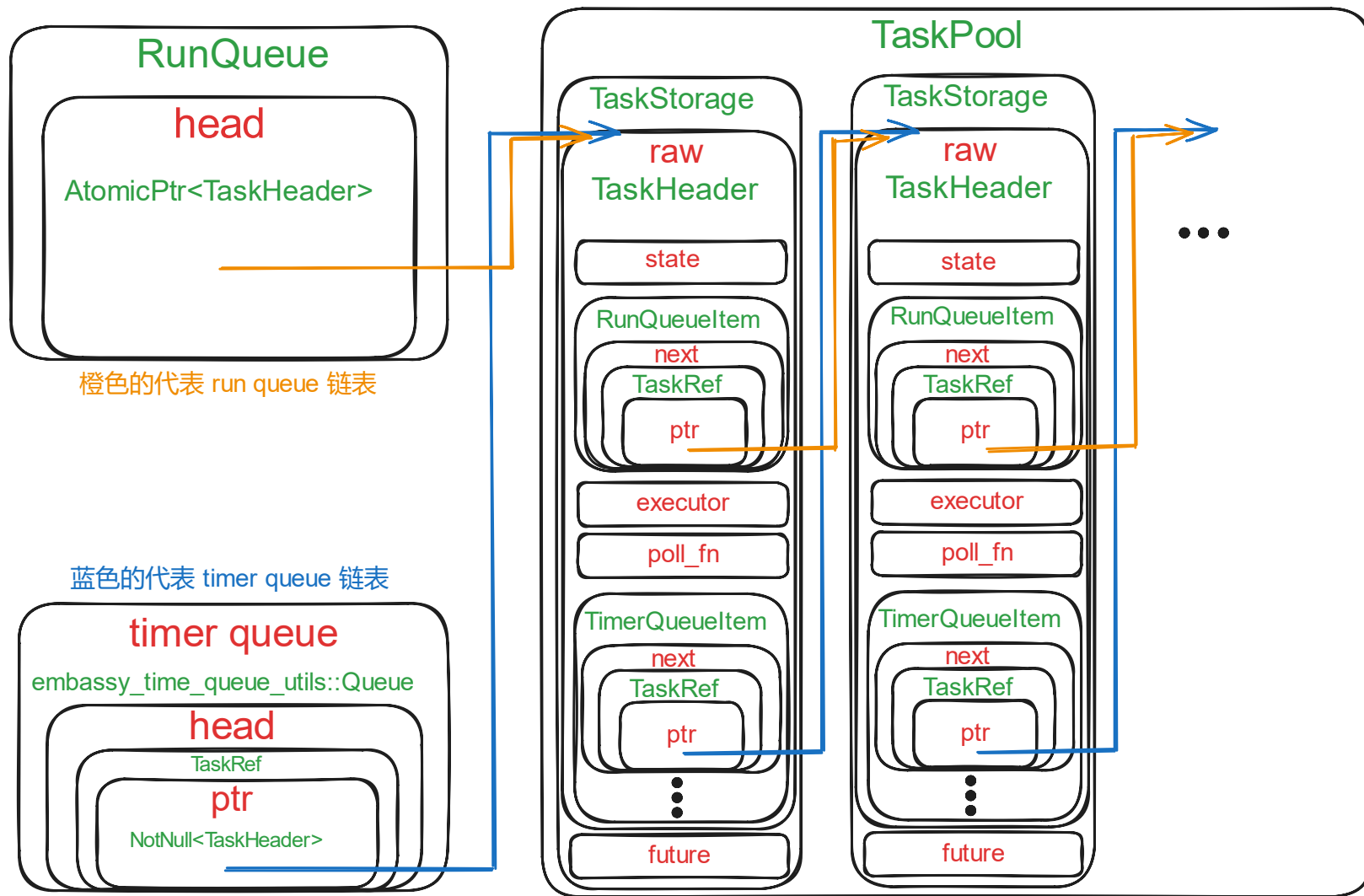
3) embassy 主要数据结构之间的转换

3.1) Waker 和 TaskRef 可以相互转换

3.2) TaskRef 存储一个指向 TaskHeader 的指针

3.3) TaskHeader 存储任务元信息, 存储在 TaskStorage 中, 是其第一个字段 raw

3.4) TaskPool<F,N> 是一组 N 个 TaskStorage<F>, 每个 TaskStorage<F> 是一个可容纳 1 个任务实例的槽位



embassy 的双链表: timer queue 和 RunQueue

实验步骤与目前进展

目前进展: Embassy

4) 基于中断的 executor

4.1) `#[interrupt]` 宏使用的其实是 `cortex_m_rt::interrupt`

4.2) `SWI_IRQ_1` 中断到来时, 会检查中断向量表中 `SWI_IRQ_1` 符号对应的函数跳转地址, 即

`__cortex_m_rt_SWI_IRQ_1_trampoline()`, 再去执行

`__cortex_m_rt_SWI_IRQ_1()` 执行 `InterruptExecutor::on_interrupt()`

4.3) `interrupt::SWI_IRQ_1`; 这行代码是为了检查当前芯片/HAL 是否真的有这个中断名称, 没有会编译错误

```
// 宏展开前
#[interrupt]
unsafe fn SWI_IRQ_1() {
    EXECUTOR_HIGH.on_interrupt()
}

// 宏展开后
#[doc(hidden)]
#[export_name = "SWI_IRQ_1"]
pub unsafe extern "C" fn __cortex_m_rt_SWI_IRQ_1_trampoline() {
    __cortex_m_rt_SWI_IRQ_1()
}

unsafe fn __cortex_m_rt_SWI_IRQ_1() {
    interrupt::SWI_IRQ_1;
    EXECUTOR_HIGH.on_interrupt()
}
```

`#[interrupt]` 宏展开前后代码

实验步骤与目前进展

目前进展：异步 timer 开发

1) Waker 设计：已完成

1.1) 直接复用 embassy-executor 的 waker 实现

1.2) Waker::wake() -> wake_task() 只负责修改当前任务状态为 TaskState::Ready

2) Task 设计：已完成

2.1) 复用 embassy 的 TaskRef

2.2) 任务状态设计发生修改：由 embassy 的按位或修改为枚举，包括 3 种状态 Ready, Running, Pending

3) Executor 设计：已完成

3.1) 维护一个存储任务的 VecDeque，任务 future 创建时插入

3.2) 执行时从队列取 TaskState::Ready 的任务 poll，修改为 TaskState::pending，返回 Poll::Pending 则放回，返回 Poll::Ready 则踢出

4) Future 设计：进行中

4.1) 实现一个 Timer Future 将 timer 逻辑放到其 poll() 中

03.7

问题与拟解决方案

问题与拟解决方案

问题：为什么不在 rCore-N 中直接使用 Embassy?

基于中断的 executor: #[interrupt] 宏底层中断基于 Vectored 模式
rCore-N 底层中断基于 Direct 模式，无法直接使用

解决：可以借鉴 Embassy 的可取之处自行设计逻辑

Embassy 的参考意义主要在于：

- 1) 状态转移和唤醒过程：Waker::wake() -> wake_task() 调用 SyncExecutor::enqueue() 调用 __pender() 函数触发软件中断（如 cortex_m armv6m 上通过 NVIC::pend(irq) 接口）执行 InterruptExecutor::on_interrupt() 进行 Executor::poll() -> SyncExecutor::poll()
- 2) 数据结构之间的联系：Waker 和 TaskRef 可以相互转换，通过 Waker 获取 TaskRef 访问 TaskHeader 任务元数据以及存储在 TaskHeader 中的 timer queue 和 RunQueue 链表节点

问题与拟解决方案

问题：rCore-N 异步 timer 开发如何引入基于中断的 Executor?

embassy 官方没有为 RISC-V 支持基于中断的 executor

解决：一种自行实现的方案

1) 在时钟中断中只做简单的事进行

wake 修改 TaskState, 然后

[sip::set_ssoft\(\)](#) 触发软件中断

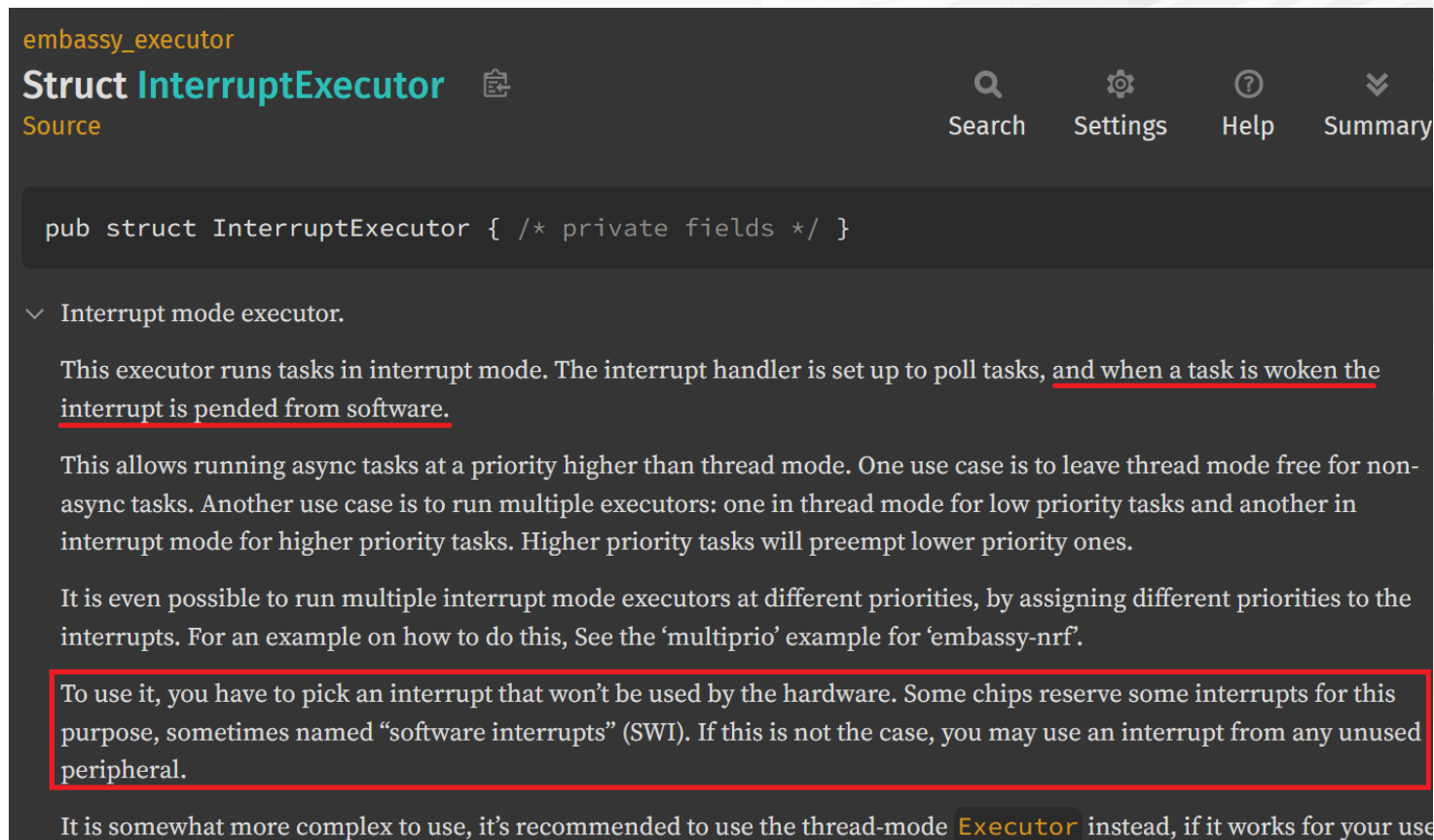
(embassy 官方也这么推荐)

2) rCore-N 中目前没有地方调用

[sip::set_ssoft\(\)](#) 因此 trap_handler

中软件中断没有被别的地方使用, 适合将

executor poll 执行逻辑放到这里



The screenshot shows the documentation for `embassy_executor` on docs.rs. The page title is `Struct InterruptExecutor`. Below the title, there is a code block showing the definition of `InterruptExecutor` as a public struct with private fields. A dropdown menu is open, showing the section `Interrupt mode executor.`. The text in this section explains that this executor runs tasks in interrupt mode, where the interrupt handler polls tasks and pends interrupts from software. It also mentions that this allows running async tasks at a higher priority than thread mode. A red box highlights a note: "To use it, you have to pick an interrupt that won't be used by the hardware. Some chips reserve some interrupts for this purpose, sometimes named 'software interrupts' (SWI). If this is not the case, you may use an interrupt from any unused peripheral." At the bottom, it says "It is somewhat more complex to use, it's recommended to use the thread-mode `Executor` instead, if it works for your use case."

```
embassy_executor
Struct InterruptExecutor
Source

pub struct InterruptExecutor { /* private fields */ }

▼ Interrupt mode executor.

This executor runs tasks in interrupt mode. The interrupt handler is set up to poll tasks, and when a task is woken the interrupt is pended from software.

This allows running async tasks at a priority higher than thread mode. One use case is to leave thread mode free for non-async tasks. Another use case is to run multiple executors: one in thread mode for low priority tasks and another in interrupt mode for higher priority tasks. Higher priority tasks will preempt lower priority ones.

It is even possible to run multiple interrupt mode executors at different priorities, by assigning different priorities to the interrupts. For an example on how to do this, See the 'multiprio' example for 'embassy-nrf'.

To use it, you have to pick an interrupt that won't be used by the hardware. Some chips reserve some interrupts for this purpose, sometimes named "software interrupts" (SWI). If this is not the case, you may use an interrupt from any unused peripheral.

It is somewhat more complex to use, it's recommended to use the thread-mode Executor instead, if it works for your use case.
```

https://docs.rs/embassy-executor/latest/embassy_executor/struct.InterruptExecutor.html



04.

下一步研究任务

「下一步研究任务

基础工作

- 1) rCore-N 异步 timer 完成开发，并验证正确性
- 2) 在 VisionFive2 上跑通 rCore-N + 异步串口驱动独立 crate

后续工作

- 3) 开始异步块设备驱动的实现
 - 1) 实现 SD 卡闪存块设备异步驱动，支持在开发板上运行文件系统并通过相应测例集
 - 2) 流程拟定与异步串口驱动开发过程类似，也是从 QEMU 到 VisionFive2



05.7

进度安排

计划和进度安排

sTime	eTime	event
3月下旬	4月上旬	rCore-N 异步 timer 开发
4月上旬	4月中旬	rCore-N + 异步串口驱动, 模拟与上板
4月中旬	5月	块设备异步驱动, 模拟与上板
5月	6月	块设备工作结尾。写论文, 准备答辩
6月	-	进行毕业设计答辩



06.

参考资料与日志

参考资料与日志

[1] 参考：星光二(Soc,JH7110)开发板上的硬件资源

https://rvspace.org/zh/homepage/product_center_sbc

[2] 日志：rCore 学习开发记录

<https://github.com/hy-huang20/GraduationProject/tree/main/过程记录/rCore>

[3] 参考：rCore-N: <https://github.com/duskmoon314/rCore-N>

[4] 参考：rCore 引入异步运行时 embassy

<https://github.com/lighkLife/rcore-blog/issues/1>

[5] 日志：复现 rCore-N 过程记录

[https://github.com/hy-huang20/GraduationProject/blob/main/过程记录/复现过程/rCore-N\[QEMU\]/readme.md](https://github.com/hy-huang20/GraduationProject/blob/main/过程记录/复现过程/rCore-N[QEMU]/readme.md)

[6] 参考：星光二开发板上使用异步串口驱动

https://github.com/BITcyman/Rust-os-learning/blob/main/board/uart_driver.md

[7] 日志：embassy 学习记录

<https://github.com/hy-huang20/rust-learning/tree/embassy-learning/dev-log>

The background of the slide features a faded image of the Tsinghua University gate and the university's name in Chinese characters (清华大学) on a stone wall. The right side of the slide is a solid purple color.

谢谢 请各位老师批评指正

汇报人：黄昊颖 指导老师：向勇